Rule Retroactivity as a Boundary Case in the Coordination Knowledge Substrate Pattern: How Rule Revisions Are Handled Without Altering Historical Substrate State

Author: Wenxin Li (Independent Researcher)
ORCID: 0009-0004-8065-3235
Date: May 7, 2026

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize, as a standalone architectural treatment, the boundary case in which an orchestration rule is revised or replaced — articulating how CKS handles rule revisions without retroactively altering historical substrate state, and how the rule version recorded in provenance at execution time preserves the source paper's path-retraceability and reproducibility commitments across rule changes.

Abstract

Rule revision is operationally common in any deployment that uses CKS to govern coordination work over time, and the architectural treatment of how revisions interact with history is non-obvious. The wrong treatment breaks more than one commitment of the source paper at once: retroactive rule application to prior cell executions breaks path retraceability; replay against historical state using the current rule version breaks the determinism contract; retroactive application decided by a vendor, an LLM, or any process other than explicit human authoring breaks the human-governed commitment at the rule layer. The treatment rests on a single load-bearing mechanism: provenance records the *specific rule version* active at execution, and replay reads that recorded version rather than re-resolving to the current rule corpus. This note formalizes the boundary case by stating the scenario, identifying the commitments it stresses, articulating the treatment, naming the anti-patterns that violate it, and stating its operational implications and limits. It is the second of approximately fifteen Phase A6 boundary-case notes; it follows the rule conflict resolution boundary and precedes the vendor unavailable boundary.

1. Why the rule retroactivity boundary needs to be formalized as standalone

Rule revision is operationally common: deployments add rules, refine the conditions under which existing rules apply, correct defects in rule logic, and replace rules outright when policy or regulatory expectations change. The naive treatment — revising a rule changes how the rule applies, with the new application taking effect against any state the rule governs — collapses several commitments at once: it alters the historical record without an authoring event the substrate carries; it makes replay non-deterministic across rule changes; and it places the decision about retroactive application outside the human-governed authoring path. The standalone formalization names the architectural treatment that prevents the collapse: rule revisions in CKS do not retroactively alter historical substrate state; cells in present operations consult the current rule version while historical executions remain associated with the rule version active at the moment of execution; replay uses the historical rule version; and re-processing of historical state under a revised rule, when humans want it, is an explicit authoring event that creates new substrate state with new provenance, leaving the original intact.

Naming this treatment matters now for prior-art reasons that go beyond CKS. A substantial portion of the "compliance updates" and "policy management" literature describes systems in which rule changes are imposed retroactively on historical events as the default behavior; the CKS treatment is the inverse of that posture, and stating it precisely as a standalone boundary case fixes its position in the public record. Together with the rule conflict resolution boundary, this note covers the two most-common stress tests of the rule layer — what happens when rules disagree, and what happens when rules change.

2. The boundary case scenario, stated precisely

A rule is in force in a CKS substrate. Cell executions have run under it; their substrate writes carry provenance metadata identifying the rule that authorized them. At a later moment, the rule is revised — its conditions tightened, its actions changed, its scope narrowed, or replaced wholesale. The revision is authored by a human under the same governance commitments that authorized the original, and lands as substrate content with its own provenance.

Two readings of "the rule has been revised" are operationally available, and the architecture's commitments depend on which reading governs. The first reading is that the revised rule is the rule, full stop: cells looking up "the rule" find the new version, replay uses the new version, and historical state is interpreted under the new version — history is re-read by the new rule rather than preserved as history under the old. The second reading is that the revised rule is an authored event in substrate history: cells in the present consult the new version, but historical executions remain associated with the rule version active at the moment of execution, and the temporal dimension of authority — which rule was in force when — becomes a property the substrate carries at the cell-execution layer.

What makes the boundary non-obvious is that both readings yield internally consistent behavior in normal operation, and the deployments most likely to encounter it — corrections workflows, regulatory recomputations, refinements of business definitions — often arrive with an intuition pointing toward the first reading ("now we know better, the new rule is the rule"). The second reading is the one the source paper's commitments require.

3. Which architectural commitments the boundary stresses

The boundary stresses five commitments at once, and the treatment must satisfy all five together.

Path retraceability. Retraceability holds if the substrate, as it currently stands, lets a reader reconstruct the path from any piece of content back to its antecedents under the conditions in force when the path was traversed. Retroactive rule application breaks this: the substrate now contains content whose recorded authorizing rule is not the rule that actually authorized it, and the audit trail no longer matches what was authored.

The determinism contract. The contract's write-determinism guarantee requires that the same substrate state plus the same orchestration rules yield equivalent cell-level behavior. If "the same rules" silently means "the rules as they currently stand," determinism across rule changes is unenforceable. The contract holds only if "the same rules" means the rules in force at the moment of the original execution, recorded as substrate content.

The authorizing-rule provenance field. A generic rule identifier — "rule R" — without version specificity is insufficient: when R is revised, the historical reference now points at a different rule than the one that authorized the historical write, and the field's purpose collapses. The architecturally load-bearing recording is the specific rule version active at the moment of the write, addressable in the substrate at any later moment regardless of subsequent revisions.

Reproducibility through replay. Reproducibility is the composition of retraceability and the determinism contract: replay over historical state, using the rules and authorizing context recorded in provenance, produces the writes the original execution produced. If replay uses the current rule version rather than the historical version, reproducibility fails even when retraceability and determinism individually appear to hold — the failure is in the integrative property, not in either commitment alone. Rule retroactivity is the canonical case in which retraceability and determinism, satisfied separately, fail together if the relationship between them across time is not preserved.

Human-governed authority over rules. Retroactive re-processing is an authoritative action, and the architecture commits human authority over rule authoring, including any rule whose effect is to re-evaluate historical state. Automatic, vendor-managed, or LLM-decided retroactivity moves that authority outside the authoring path the human-governed commitment names.

These commitments stress different facets of the same operational requirement: rule changes do not silently alter what the substrate's history says.

4. The architectural treatment

The architectural treatment is direct and decomposable into six operational moves, each derivable from existing commitments.

Prior substrate state remains unchanged when rules are revised. A rule revision is itself a write to the substrate, but it does not alter, replace, or re-interpret prior substrate content. Historical writes remain at their original addresses with their original content and their original provenance.

The authorizing-rule provenance field records the specific rule version active at execution. Not a generic rule identifier but the specific version in force at the moment of the cell execution. The version is itself substrate-resident authoritative content with its own provenance, addressable at any later moment regardless of subsequent revisions.

Cells in present operations consult the current rule version. New cell executions look up the rule corpus as it currently stands and operate under the current version, which they record in their own provenance. From the cell's perspective there is no ambiguity: "the rule" is what the substrate currently says it is. The temporal dimension is captured only in provenance.

Replay uses the historical rule version. Reproducibility through replay is achieved by replay reading the rule version recorded in the historical provenance, not the current rule version. This is what makes replay deterministic across rule changes: the same historical state plus the same historical rule version yields the same writes the original execution produced, regardless of how many revisions have intervened.

Re-processing under a new rule is itself an authored event. A human authors a rule whose effect is to re-process specified historical state under the new rule's logic, and a cell executes that rule. The cell's writes are new substrate content with new provenance; the original historical state and provenance remain unchanged. The trace now contains both — the original content and the re-processed content — each addressable in its own right, each with its own authorizing rule version recorded in provenance.

Rule versioning is native to the architecture. No separate versioning infrastructure is required. Each rule revision is itself authored as a substrate write with its own provenance; the rule corpus carries its own history at the same architectural layer that the rest of the substrate carries history. The treatment does not commit to any specific representation — addressing rules by content hash, sequence number, named version, or any other addressable form is a deployment decision.

The treatment also distinguishes retroactive re-processing from override. Override is a human acting on specific substrate state, with no rule needed to authorize it. Retroactive re-processing is a human authoring a rule whose effect is to re-evaluate categorical historical state under new logic. Both are legitimate; the difference between specific-state correction and categorical re-evaluation matters for how each is recorded and how each is later read.

5. Anti-pattern treatments that violate the architecture

Six anti-patterns name the most operationally available ways to handle rule revision that violate the CKS treatment.

Rule-update-retroactively-rewrites-history. A rule revision propagates back through the substrate, modifying historical writes to reflect the new rule's logic. Violates path retraceability and the authorizing-rule provenance field. The most overtly history-altering anti-pattern, and consequently the easiest to detect.

Automatic-re-processing-on-rule-change. A rule revision triggers automated re-evaluation of prior substrate state without an explicit authoring event. The re-processing may produce results that look CKS-coherent, but no human-authored rule is on record as having directed it, and no cell execution

carrying its own provenance is on record as having performed it. Violates the human-governed and provenance commitments.

Rule-update-silently-changes-interpretation. The rule revision does not alter historical writes, but later reads of historical substrate state are interpreted under the new rule rather than the rule recorded in provenance. Violates the determinism contract: the read is silently re-resolving to the current corpus. The quieter sibling of the first anti-pattern, and the one most likely to escape detection.

Vendor-managed-retroactivity. The substrate's hosting environment, a vendor's rule-management product, or a runtime middleware layer determines whether a rule revision applies retroactively. Violates the human-governed commitment at the rule layer: the architecture commits authoritative actions to the human-governed authoring path, not to a vendor's policy path.

LLM-mediated-retroactive-application. An LLM is consulted at read or replay time to decide whether and how a rule revision applies to historical state. Violates the AI-as-substrate-mediator commitment (the LLM is a mediator, not an authority over rule applicability) and the determinism contract (LLM-mediated decisions are non-deterministic).

Single-rule-identifier-without-versioning. The authorizing-rule provenance field is populated with a generic rule identifier rather than a version-specific reference. Violates reproducibility directly: replay using "rule R" resolves to whatever the current R is, which may not be the R that was in force at the historical execution. The failure can be silent — replay produces results, just different ones — and is detectable only by the replay-divergence signal stated in §8.

A system exhibiting any of the six is not CKS-coherent on the rule retroactivity axis; the naming matters because each anti-pattern is remediable specifically once identified.

6. Operational implications

Three implications follow directly from the treatment.

Rule revisions are substrate changes like any other. A rule revision is authored as a substrate write with its own provenance; rule corpora carry their own history at the same architectural layer that other substrate content carries history. No separate rule-management subsystem is required, and any subsystem that claims to manage rules outside this apparatus is a candidate for the vendor-managed-retroactivity anti-pattern.

Cells operate naturally under current rules. A cell executing in the present consults the current version. Cells do not have to reason about which version of a rule applies; the architecture binds that question at write time.

Re-processing is an authored, recorded event. Deployments that need to re-evaluate historical state under a revised rule do so by authoring a re-processing rule. The result is new substrate content alongside the original — never replacing it — and the trace reads as a richer history rather than as a substituted one. Auditors and downstream readers can distinguish the original record from the re-processed record by reading the substrate alone.

7. Limits of the architectural treatment

The treatment is bounded in four ways.

It applies to rule-behavior revisions, not to object-level state changes. Direct edits to substrate content under the modify and override rights are addressed by those commitments and by the provenance their writes carry. The retroactivity boundary is concerned specifically with revisions to the orchestration rules that govern cell behavior.

It does not address authority distribution changes. Changes in which humans hold which rights over which substrate scopes raise their own boundary case, addressed in a later Phase A6 note. The retroactivity treatment assumes the human-governed authority architecture is stable across the rule revision.

It does not address composition pattern changes. Revisions to how substrates compose with adjacent AI components are properly the subject of a Phase A4 composition-pair note rather than a Phase A6 boundary-case note.

It does not require rules to be unchangeable. Rules may be revised freely; the architecture supports revision as authored substrate content. The treatment specifies what happens to history when revisions occur, not whether revisions occur.

8. Operational test

A system implements the rule retroactivity treatment if and only if all of the following are true at all times during the substrate's existence:

1. **History is unaltered by rule revisions.** A rule revision authored after a historical cell execution does not modify the historical write's content or its provenance.
2. **The authorizing-rule provenance field records a version-specific rule reference.** Each cell-mediated write records the specific rule version in force at the moment of the write.
3. **Cells in present operations consult the current rule version, and replay consults the historical rule version.** Two cells executing the same logic over the same substrate state at different moments — one before a rule revision, one after — operate under different rule versions, each recorded in their own provenance. Replay uses the rule version recorded in the corresponding provenance.
4. **Re-processing under a revised rule is an authored event with its own provenance.** Re-evaluation of historical state under a revised rule is performed by a cell executing under a human-authored rule, and the cell's writes are new substrate content with new provenance. Original historical state and provenance remain unchanged.
5. **The replay-divergence signal holds.** Replay using the rule version recorded in provenance produces the writes the original execution produced; replay using a current (post-revision) rule version may produce different writes. The divergence is the architectural signal that the historical version, not the current, is the one reproducibility binds.

In one sentence: a CKS substrate handles rule revisions correctly when, and only when, the rule version recorded in each cell execution's provenance is what governs replay of that execution, regardless of

how the rule corpus has changed since. A system that fails any of (1)–(5) may handle rule revisions in some other principled way, but is not CKS-coherent on the rule retroactivity axis.

9. Why naming the boundary as standalone matters

Rule retroactivity is the boundary case most easily mishandled in practice. Rule revision is operationally common; the temporal dimension of "what rules apply" is operationally non-obvious; the most operationally available ways to handle rule revision — propagate through history, re-resolve to the current corpus at read time, let the vendor decide — happen to be the ones that violate the architecture. Naming the boundary as a standalone treatment makes the architecturally correct handling specifiable, testable, and citable as something a system either does or does not implement. The standalone framing also pins the distinction between rule revision and override in the right place: both are legitimate and CKS-coherent under their respective commitments, but conflating them produces architectures in which override and re-processing become indistinguishable in the trace, and the trace loses the property that makes it reconstructible — the ability to read what each event was, not only that it occurred.

This note is the second of approximately fifteen Phase A6 boundary-case notes. The rule conflict resolution boundary and the rule retroactivity boundary together address the two most-common stress tests of the rule layer. The remaining Phase A6 notes will cover boundary cases at other layers: vendor unavailable, LLM consultation timeout, composition partner failure, and beyond. Subsequent work that handles rule revision differently than this note describes is using a different architecture, and the difference should be named.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems.* April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Rule Retroactivity as a Boundary Case in the Coordination Knowledge Substrate Pattern: How Rule Revisions Are Handled Without Altering Historical Substrate State.* May 7, 2026. ORCID: 0009-0004-8065-3235.